

Multiple Linear Regression: Geometric Intuition and Code

Contents

1. Overview	2
2. From a line to a hyperplane	3
2.1 Two inputs: a plane	3
2.2 More inputs: a hyperplane	4
3. The equation	4
3.1 What the coefficients mean	5
4. Multiple linear regression in scikit-learn	6
5. Summary	8
6. Sources	8
7. Key terms	8

Where this fits: Pipeline map steps 0 (Foundations), 8 (Model). See the [Course map](#).



- **Builds on:** Regression problems ([Note 3](#)); Simple linear regression ([Note 50](#)); Normal equation ([Note 54](#)); Batch gradient descent ([Note 58](#)); Dot product ([Note 362](#)).
- **Leads to:** Polynomial regression ([Note 61](#)); Ridge regression ([Note 63](#)); Perceptron trick ([Note 70](#)); Support vector machines ([Note 92](#)); Perceptron ([Note 1004](#)).
- **Compare with:** ANN for regression ([Note 1013](#)).

1. Overview

Key point: Multiple linear regression is simple linear regression with more than one feature. With two inputs it fits a plane; with more, a hyperplane.

A **feature** (G-772) is an input variable (one column of the data table), the **target** (G-1949) is the output we predict, and an **observation** (G-1374) is one record (one row). Figure 1 marks all three on the first rows of this Note's example data.

row	feature 1 (x_1)	feature 2 (x_2)	target (y)
1	0.27	-0.52	32.6
2	-1.43	0.5	-75.3
3	-0.1	1.7	56.2
4	-0.28	-1.54	-97
5	-0.19	0.7	-103.4

green row: one observation (one student, one house, one record)

Figure 1: The first five observations of the example data. Blue columns: the two features. Orange column: the target. Green row: one observation.

Simple linear regression used one feature, CGPA, to predict the package. Real data almost always has several features: CGPA, gender, IQ, 12th-grade marks, and so on. Linear regression with more than one feature is called **multiple linear regression** (G-1279).

Nothing new has to be learned for multiple linear regression: everything from simple linear regression carries over. **Simple linear regression** (G-1808) is just the special case with one feature. A recipe works the same way: the cake's taste depends on sugar, flour and butter together, each in its own amount, instead of on sugar alone. This Note covers the geometry and the code; the next two Notes derive the mathematics and code it from scratch.

2. From a line to a hyperplane

Key point: One input: a line in 2D. Two inputs: a plane in 3D. n inputs: a hyperplane in $n + 1$ dimensions.

2.1 Two inputs: a plane

Key point: With CGPA and IQ as inputs, the data lives in 3D, and the model is a flat plane through the cloud of points.

Take two inputs, CGPA (x_1) and IQ (x_2), and the package (y) as output. Each student is now a point in 3D: CGPA along one axis, IQ along another, package upwards.

In 2D we drew the line that passes closest to all points. In 3D we draw a flat **plane** (G-1502) that cuts through the cloud of points: some points lie above it, some below, and the plane keeps as close as possible to all of them (Figure 2).

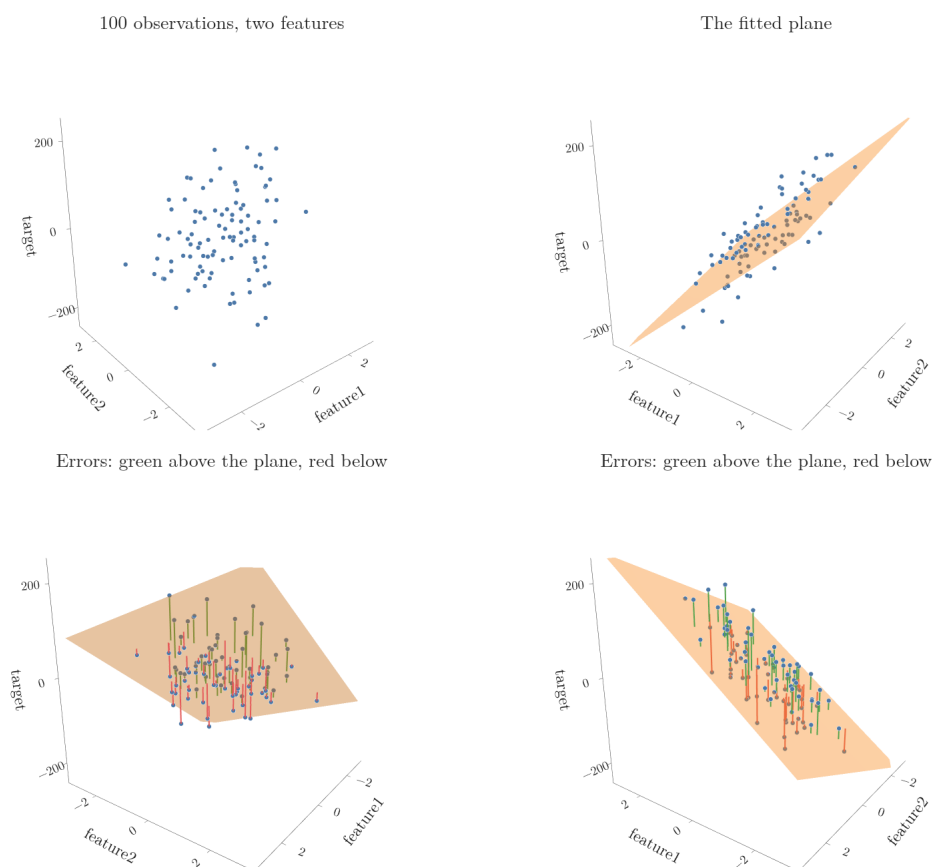


Figure 2: 100 observations in 3D, then the fitted plane, then each observation's error as a stick to the plane (green above, red below), while the view circles once

Watch the sticks as the view turns: about half the points sit above the plane (50 of 100) and half below, and from the side the plane cuts through the middle of the cloud. Fitting the

plane means making these sticks, squared and added up, as small as possible, just as with the line.

2.2 More inputs: a hyperplane

Key point: Beyond three dimensions we cannot draw the picture, but the equation keeps the same form.

With three inputs the data is 4-dimensional, and the model is the 4D version of a plane. A flat surface in more than three dimensions is called a **hyperplane** (G-911). We cannot draw it, but the mathematics works exactly the same (Figure 3).

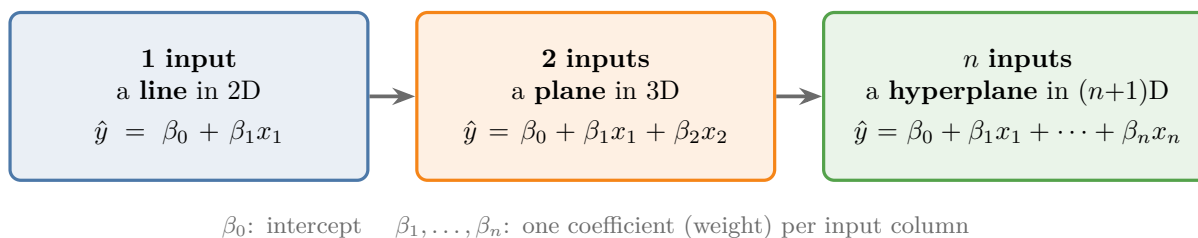


Figure 3: From a line to a hyperplane

3. The equation

Key point: The prediction is the intercept plus one coefficient times each input: $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$.

For simple linear regression we wrote $y = mx + b$. With more inputs, letters run out, so we rename the numbers with the Greek letter beta: β_0 for the intercept and $\beta_1, \beta_2, \dots$ for the slopes.

In words: start from the intercept, then add each input multiplied by its own coefficient.

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

With numbers, from the model in Figure 2:

$$\hat{y} = -1.9 + 58.6x_1 + 29.1x_2$$

For a point with $x_1 = 1$ and $x_2 = 0.5$: $\hat{y} = -1.9 + 58.6 + 14.6 = 71.3$. Figure 4 builds this prediction one term at a time: start at the intercept, then let each feature add its own share.

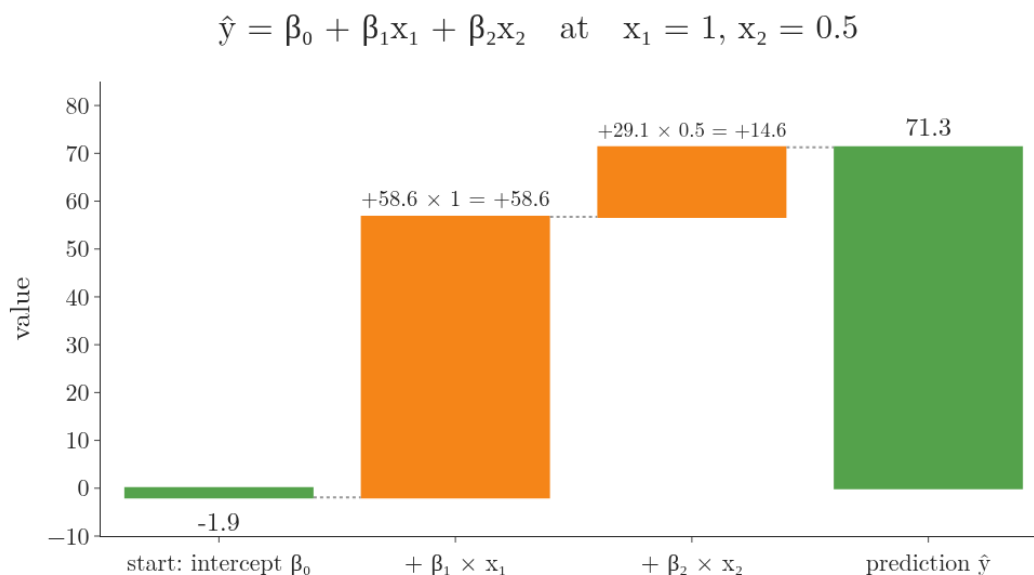


Figure 4: One prediction built term by term: the intercept -1.9 , plus 58.6×1 , plus 29.1×0.5 , gives 71.3 .

With n features there are $n + 1$ numbers to find: one **coefficient** (G-407) per feature plus the **intercept** (G-960). Training the model means finding them.

3.1 What the coefficients mean

Key point: Each coefficient is the change in the output when its feature rises by 1 and the other features stay the same: the feature's weight.

The meaning carries over from the slope m :

- β_1 : how much the prediction changes when x_1 rises by 1 and every other input stays fixed.
- β_2 : the same for x_2 , and so on.
- β_0 : the prediction when every input is 0.

Figure 5 slices the plane of Figure 2 at three fixed values of feature2. Each slice is a line with slope 58.6, and the slices sit 29.1 apart: one coefficient per direction.

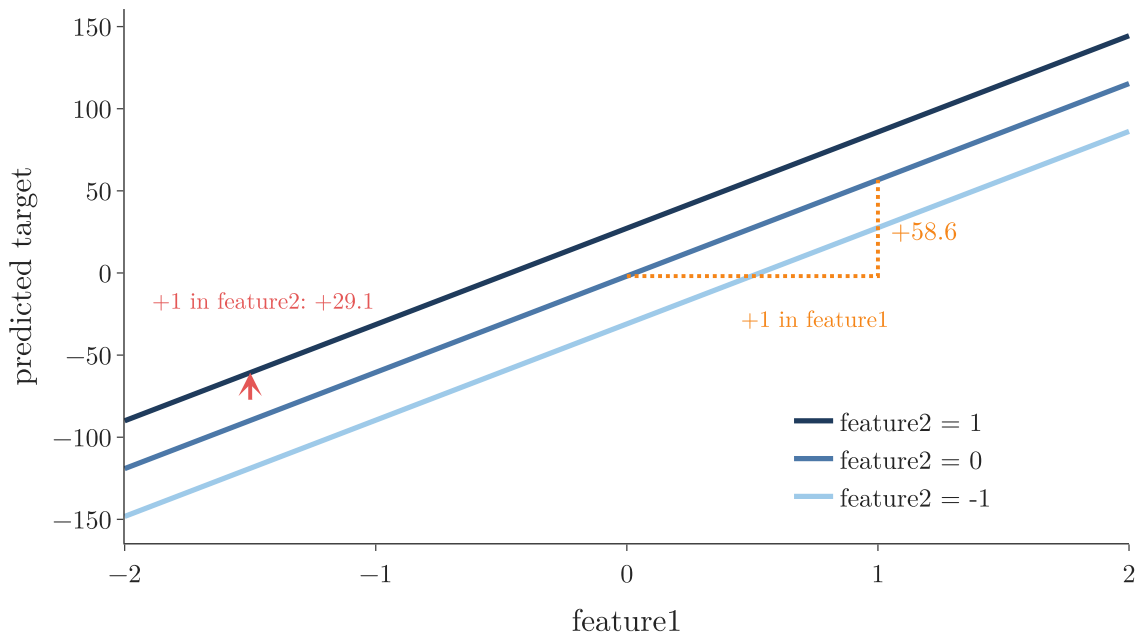


Figure 5: Slices of the fitted plane at feature2 = −1, 0 and 1. One step along feature1 adds 58.6 (orange); one step in feature2 lifts the whole line by 29.1 (red)

So the coefficients act as **weights** (G-2111): in Figure 2, the target depends about twice as strongly on feature 1 (58.6 per unit) as on feature 2 (29.1 per unit). In the placement example, β_1 would say how much the package depends on CGPA and β_2 how much on IQ. A coefficient near 0 means the target hardly depends on that feature.

Extra: Comparing coefficients only makes sense when the inputs are on similar scales. A coefficient of 58.6 per unit of CGPA and 0.05 per IQ point says nothing about which matters more, because one IQ point is a much smaller step than one CGPA point. **Standardising** (G-1874) the inputs first puts all coefficients on the same footing (Gelman 2008). In Figure 2 the comparison is fair: both inputs from `make_regression` already have a standard deviation of about 1.

4. Multiple linear regression in scikit-learn

Key point: The code is exactly the same as for one feature; `LinearRegression` simply returns one coefficient per feature.

The example data has 100 observations, 2 features and some noise, made with scikit-learn’s `make_regression`.

Python: Training on two features.

```
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression
```

```

from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=100, n_features=2,
                      n_informative=2, noise=50, random_state=7)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=3)

lr = LinearRegression().fit(X_train, y_train)
lr.coef_      # [58.6, 29.1]: one per feature
lr.intercept_ # -1.9

```

make_regression (G-1153) invents data that follows a linear pattern plus random noise; random_state fixes it so the numbers repeat.

The metrics of the [regression metrics Note](#) are computed exactly as before: the errors are now gaps to the plane instead of gaps to the line. With several features, adjusted R^2 (section 7 of that Note) is the fairer score, because it charges a penalty for each feature.

On the 20 test observations: MAE 40.1, MSE 2614.9 and $R^2 = 0.61$. The noise added to the data limits how good any plane can be: with less noise the same code scores higher, and with no noise it fits perfectly (R^2 of 0.85 at noise=25, 1.0 at noise=0; see the Notebook). Figure 6 shows the test predictions for all three: as the noise falls, the points close in on the diagonal of perfect predictions.

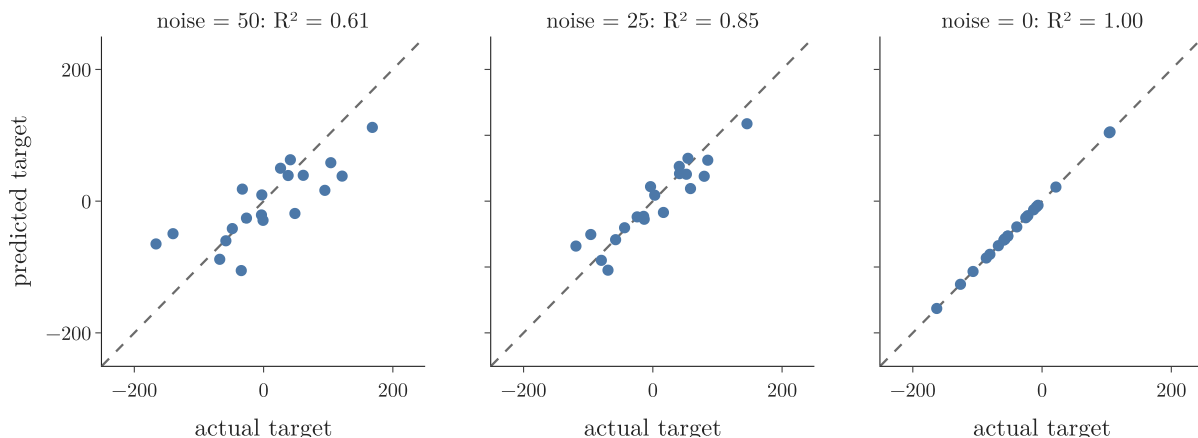


Figure 6: Predicted against actual target on the 20 test observations, for the same recipe with noise 50, 25 and 0

Extra: To draw the plane, we predict on a grid of (x_1, x_2) points and plot the predictions as a surface. The Notebook does this with Plotly, and the 3D plot can be turned with the mouse to see the points above and below the plane.

5. Summary

	Simple linear regression	Multiple linear regression
Inputs	1	2 or more
Shape of the model	line	plane (2 inputs), hyperplane (more)
Equation	$\hat{y} = mx + b$	$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n$
Numbers to find	2	$n + 1$
scikit-learn	<code>LinearRegression</code>	the same <code>LinearRegression</code>

- Real data almost always has several inputs, so multiple linear regression is the version used in practice.
- Each coefficient is the change in the output per unit of its input, with the other inputs fixed: a weight.
- The intercept β_0 is the prediction when every input is 0.

6. Sources

Built from

- CampusX, “Multiple Linear Regression | Geometric Intuition & Code”, YouTube, <https://www.youtube.com/watch?v=ashGekqstl8>
- StatQuest with Josh Starmer, “Multiple Regression, Clearly Explained!!!”, YouTube, <https://www.youtube.com/watch?v=EkAQAi3a4js> (R^2 is computed as in simple regression; adjusted R^2 for the extra parameters)

Other references

- Gelman, A. (2008). Scaling regression inputs by dividing by two standard deviations. *Statistics in Medicine* 27(15): 2865–2873.

7. Key terms

Term	Meaning
Feature	An input variable: one column of the data table
Target	The output we predict
Observation	One record: one row of the data table
Multiple linear regression	Linear regression with two or more features
Plane	A flat surface in 3D; the model for two features
Hyperplane	A flat surface in more than three dimensions; the model for three or more features
Coefficient (β_i)	The weight of one feature: the change in the output per unit of that input, others fixed

Term	Meaning
<code>make_regression</code>	scikit-learn function that generates data following a linear pattern plus noise